

Randomised Algorithms

Sahil M Deo

May 27, 2026

Contents

1	Introduction	2
1.1	What even is a Randomised Algorithm?	2
1.2	Implementation of Randomness in C++	2
2	Algorithmic Performance	4
2.1	Worst Case Analysis	4
2.2	Average Case Analysis	4
2.3	Time Complexity Analysis	5
3	Standard Examples	5
3.1	Quick Sort	5
3.2	Quick Select	8
3.3	The Dating Problem	8
3.3.1	Why Threshold Strategies are Optimal	9
3.3.2	Finding the Optimal Threshold	10
3.3.3	Behaviour for Small n	11
4	My Examples	12
4.1	Matrix Problem	12
4.1.1	Deterministic Method	12
4.1.2	Randomised Method	13
4.2	Rolling Window Multiset	14
5	Heuristic Algorithms	15
5.1	The Dominating Set Problem	15
5.1.1	A Randomised Solution	16
5.1.2	A Greedy Heuristic	18
5.2	Exam Scheduling	19
5.2.1	A Probabilistic Method Analysis	20
5.2.2	A Local Random Decision Method	21

1 Introduction

1.1 What even is a Randomised Algorithm?

A randomised algorithm is an algorithm that makes random choices during its execution. Unlike deterministic algorithms, whose behaviour is completely determined by the input, the behaviour of a randomised algorithm may vary between different executions on the same input.

Typically, the algorithm uses a random number generator (rng for short) to decide between multiple possible actions. As a result, quantities such as running time, memory usage, or even correctness¹ can become random variables (though usually not all of them at the same time).

The performance of a randomised algorithm is therefore analysed using probability and expectation. If T_{run} denotes the runtime of the algorithm, we are usually interested in quantities such as $T_{algo} = E[T_{run}]$ over all possible random choices made by the algorithm.²

Randomised algorithms are often used for the following purposes:

- avoid adversarial worst case inputs,
- simplify algorithm design,
- obtain fast approximate solutions.

1.2 Implementation of Randomness in C++

Lets make things more concrete with some actual code and implementation details! Modern C++ provides pseudo-random³ number generators through the `<random>` library. A commonly used generator is `mt19937`, which is based on the Mersenne Twister algorithm.

```
#include <bits/stdc++.h>
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count()); //using time as a seed
```

This creates an object `rng` with seed⁴ as current time which will be passed as a parameter to other functions. It has the capability to produce "truly" random 32-bit integers.

Generating Random Integers

To generate a random integer uniformly from a range $[L, R]$, we use

```
uniform_int_distribution<int>uid(L,R);
```

```
int x=uid(rng);
```

```
int y=uid(rng);
```

```
.
.
.
```

Each generation takes around constant time.

¹The fact that correctness itself may become probabilistic can initially seem undesirable, especially in critical systems where absolute correctness is expected. However, in many practical situations, a sufficiently small error probability is considered acceptable. For example, if an algorithm can be shown to fail with probability at most 10^{-30} , then the chance of failure may already be smaller than the probability of random hardware faults such as memory bit flips caused by cosmic radiation, which programmers typically ignore during ordinary software development.

² T_{algo} can still be a random variable over the input - so this may **not** be the same as average case performance of the algorithm T_{avg} , which is an expected value over all possible inputs.

³Since computers are deterministic, they rely on parameters like atmospheric noise and temperature for randomness. The details are irrelevant for us... We will take pseudo random as true random for our purposes

⁴seed values are used in pseudo random generators as a way to recreate the generated random numbers. This can be used for debugging a randomised algorithm.

Generating Random Real Numbers

To generate a real number uniformly from an interval $[x, y)$, we use

```
uniform_real_distribution<double>urd(x,y);
```

```
double x=urd(rng);
double y=urd(rng);
.
.
.
```

This also takes $\Theta(1)$ time per generation.

Generating a Random Permutation

A random permutation of an array (in-place modification) can be generated using `shuffle`:

```
shuffle(a.begin(),a.end(),rng);
```

Internally, this does something like:

```
for(int i=n-1;i>0;i--)
{
    int j=random integer from [0,i];
    swap(a[i],a[j]);
}
```

(Try to think why all permutations are equally likely with this method!)

Implementing Random Decisions

Sometimes, an algorithm must choose between multiple actions with specified probabilities. For example, let three actions X,Y,Z be such that they are executed with probabilities:

$$\Pr(X) = \frac{1}{5}, \quad \Pr(Y) = \frac{3}{5}, \quad \Pr(Z) = \frac{1}{5}$$

One simple way to implement this is to generate a random integer uniformly from 1 to 5:

```
uniform_int_distribution<int>uid(1,5);
```

```
int v=uid(rng);
```

```
if(v==1)
{
    //do X
}
else if(v<=4)
{
    //do Y
}
else
{
    //do Z
}
```

thus implementing the required probabilities.

2 Algorithmic Performance

For algorithms whose performance⁵ depends on the input size or other input parameters, we usually estimate the number of elementary operations performed as a function of the input size n or the maximum value of input A_{max} , usually denoted by A . This gives a mathematical way to compare algorithms independent of hardware or implementation details.

A classical example is Bubble Sort. The algorithm repeatedly traverses the array, comparing adjacent elements and swapping them whenever they are in the wrong order. After each pass, the largest remaining element moves to its correct position near the end of the array.

With some thought it can be proved that the total number of swaps performed by Bubble Sort is exactly equal to the number of inversions N_{inv} in the input array. (An inversion is a pair of indices (i, j) such that $i < j$ but $a_i > a_j$.) By taking swaps to be one operation taking constant time, we can estimate runtime of Bubble sort to be $T_{run} \propto N_{inv}$

2.1 Worst Case Analysis

In worst case analysis, we determine the maximum possible number of operations over all valid inputs of size n . For Bubble Sort, the worst case occurs when the array is sorted in reverse order, where every pair is inverted.

$$N_{inv,max} = \binom{n}{2} = \frac{n(n-1)}{2}$$

Worst case analysis is useful because it guarantees an upper bound on the running time regardless of the input provided.

2.2 Average Case Analysis

In average case analysis, we compute the expected number of operations assuming that all valid inputs are equally likely.

For Bubble Sort, this corresponds to analysing the expected number of inversions in a random permutation of size n . For every pair of indices (i, j) with $i < j$, the probability that the pair forms an inversion is

$$\Pr((i, j) \text{ is an inversion}) = \frac{1}{2}$$

Define the indicator variable:

$$I_{ij} = \begin{cases} 1, & \text{if } (i, j) \text{ is an inversion} \\ 0, & \text{otherwise} \end{cases}$$

$$N_{inv} = \sum_{1 \leq i < j \leq n} I_{ij}$$

$$\mathbb{E}[N_{inv}] = \mathbb{E} \left[\sum_{1 \leq i < j \leq n} I_{ij} \right] = \sum_{1 \leq i < j \leq n} \mathbb{E}[I_{ij}]$$

$$\mathbb{E}[I_{ij}] = 1 \cdot \frac{1}{2} + 0 \cdot \frac{1}{2} = \frac{1}{2}$$

⁵Although this section focuses primarily on running time, the same methods of analysis can also be applied to other resources such as memory usage.

Therefore,

$$\mathbb{E}[N_{\text{inv}}] = \sum_{1 \leq i < j \leq n} \frac{1}{2} = \frac{1}{2} \binom{n}{2} = \frac{n(n-1)}{4}$$

So notice that if we randomise the input sequence, T_{algo} becomes equal to T_{avg} thus improving performance by a factor⁶ of 2.

2.3 Time Complexity Analysis

We are sometimes interested in comparing runtimes like $T_{\text{run}} = n^2$ to $T_{\text{run}} = 100n \log n$. To compare growth rates, we use asymptotic notation:

$$T(n) = \Theta(f(n))$$

means that $T(n)$ grows asymptotically at the same rate as $f(n)$.

My favourite way to think about time complexity is as follows: we try to find the “simplest” function $f(n)$ such that

$$\lim_{n \rightarrow \infty} \frac{T(n)}{f(n)} = k, \quad k \in \mathbb{R}_{>0}$$

That is, $T(n)$ and $f(n)$ grow at the same asymptotic rate up to a positive constant factor.

For example, Bubble Sort performs on the order of n^2 operations in both the average and worst case, so its time complexity is

$$T(n) = \Theta(n^2)$$

Asymptotic analysis allows us to compare algorithms by their rate of growth rather than implementation-dependent constants.

This allows us to say that A: $T_{\text{run}} = n^2$ is worse than B: $T_{\text{run}} = 100n \log n$ since A has $T(n) = \Theta(n^2)$ and B has $T(n) = \Theta(n \log n)$ even though for some small n the first one may well be better, but for small n the performance doesn't really matter...

3 Standard Examples

3.1 Quick Sort

Quick Sort is one of the most important examples of a randomised algorithm in practice. The basic idea is very simple:

1. Choose a pivot element.
2. Partition the array into:
 - elements smaller than the pivot,
 - elements equal to the pivot,
 - elements greater than the pivot.

This takes n comparisons with our pivot

3. Recursively sort the smaller and larger parts.

⁶Randomising the input itself takes about n operations, so more rigorously we would say an asymptotic factor of 2. In most cases we will be able to neglect the time required to randomise the input.

A deterministic implementation may always choose the first or last element as pivot. Unfortunately, this can lead to very poor performance on adversarial inputs. For example, if the array is already sorted and we always choose the first element as pivot, the recursion becomes extremely unbalanced:

$$(n-1) + (n-2) + \dots + 1 = \Theta(n^2)$$

Thus deterministic Quick Sort has worst case complexity

$$T(n) = \Theta(n^2)$$

The standard randomised version instead chooses the pivot uniformly at random.

```
uniform_int_distribution<int>uid(0,a.size()-1);
int pivot=uid(rng);
swap(a[pivot],a[r]);
```

This tiny modification dramatically changes the behaviour of the algorithm. Intuitively, it becomes very unlikely that we repeatedly choose extremely bad pivots.

Runtime Analysis

Let $T(n)$ denote the expected runtime of randomised Quick Sort on an array of size n over all possible inputs.

Suppose the pivot finally ends up at position k , where $0 \leq k \leq n-1$. Then the recursive subproblems have sizes k and $n-1-k$. Since partitioning itself takes n operations, we obtain the recurrence

$$T(n) = T(k) + T(n-1-k) + n$$

Because the pivot is chosen uniformly at random, every value of k is equally likely. Taking expectation over all random choices made by the algorithm gives

$$T(n) = n + \frac{1}{n} \sum_{k=0}^{n-1} (T(k) + T(n-1-k))$$

By symmetry,

$$\sum_{k=0}^{n-1} T(k) = \sum_{k=0}^{n-1} T(n-1-k)$$

therefore

$$T(n) = \Theta(n) + \frac{2}{n} \sum_{k=0}^{n-1} T(k)$$

$$S(n) = \sum_{k=0}^n T(k)$$

Then

$$T(n) = S(n) - S(n-1)$$

and the recurrence becomes

$$S(n) - S(n-1) = cn + \frac{2}{n}S(n-1)$$

$$S(n) = \left(1 + \frac{2}{n}\right)S(n-1) + cn$$

$$\frac{S(n)}{(n+1)(n+2)} = \frac{S(n-1)}{n(n+1)} + \frac{cn}{(n+1)(n+2)}$$

Summing this recurrence telescopically gives

$$\frac{S(n)}{(n+1)(n+2)} = \Theta\left(\sum_{k=1}^n \frac{1}{k}\right) = \Theta(\log n)$$

Therefore,

$$S(n) = \Theta(n^2 \log n)$$

Finally,

$$T(n) = S(n) - S(n-1) = \Theta(n \log n)$$

Hence the expected runtime of randomised Quick Sort is

$$T(n) = \Theta(n \log n)$$

This analysis demonstrated a powerful technique to find the expected runtime - recursion on input size. It was a bit messy and lengthy, but sometimes yields results very quickly.

Another possible way is through indicator variables. Observe that any pair of elements is compared at most once during the entire execution of Quick Sort.

Let

$$I_{ij} = \begin{cases} 1, & \text{if elements } i, j \text{ are compared} \\ 0, & \text{otherwise} \end{cases}$$

Then the total number of comparisons is

$$C = \sum_{1 \leq i < j \leq n} I_{ij}$$

Two elements are compared only if one of them becomes the first pivot chosen among all elements lying between them in sorted order. It can be shown that

$$\Pr(I_{ij} = 1) = \frac{2}{j-i+1}$$

Therefore,

$$\mathbb{E}[C] = \sum_{1 \leq i < j \leq n} \frac{2}{j-i+1} = \Theta(n \log n)$$

Hence randomised Quick Sort has expected complexity

$$T(n) = \Theta(n \log n)$$

3.2 Quick Select

The Problem: find the k -th smallest element of an unsorted input array

An obvious method is to sort the array using Merge Sort in $\theta(n \log n)$ and just access the k -th element - this is overall $\theta(n \log n)$ and is deterministic. Then how could randomness help?

Quick Select is an algorithm used to find the k -th smallest element of an array.

Similar to Quick Sort, the algorithm chooses a pivot and partitions the array into three categories by doing $n-1$ comparisons between the pivot and every other element as before:

- elements smaller than the pivot,
- elements equal to the pivot,
- elements greater than the pivot.

However, unlike Quick Sort, we recurse into only **one** side:

- if the pivot ends up at position k , we are done,
- if k lies to the left, recurse left,
- otherwise recurse right.

Thus each step discards a significant fraction of the array. This is a similar approach to Binary Search.

Expected Runtime

Suppose partitioning takes n time where array size is n . Let $T(n)$ be the runtime. If the pivot is the i -th smallest element, then

$$T(n) = n +$$

which is a geometric series:

$$T(n) = \Theta(n)$$

Thus randomised Quick Select runs in expected linear time.

3.3 The Dating Problem

Problem Statement: Consider the problem ⁷ of finding the best partner. Assume that the number of people you will eventually date is fixed, i.e. some known value n . The people appear in a uniformly random order. After dating a person, you must immediately decide whether to commit to them forever or reject them permanently and continue searching. Figure out a strategy that maximises the probability of ending up with the best partner.⁸

We assume that while dating person i , we only know how they compare relative to the people we have already dated. In particular, we can determine whether they are the best person seen so far.

Most sources directly jump to the famous strategy of:

“Reject the first r people, then accept the first person afterwards who is better than everyone seen so far.”

⁷Although this problem belongs more to optimal stopping theory rather than randomized algorithms, it illustrates several recurring themes in probabilistic algorithm design

⁸This is a different problem from maximising the expected “value” of your partner. Here the objective is specifically to maximise the probability of selecting the single best partner among all n people.

However, they often omit the proof of why an optimal strategy must necessarily have this threshold form. We first prove this fact.

3.3.1 Why Threshold Strategies are Optimal

Suppose we are currently considering the k -th person.

If the current person is *not* the best among the first k people seen so far, then accepting them can never succeed: This is because there already exists an earlier person who is strictly better, and that earlier person has already been rejected. Hence an optimal strategy should only ever consider accepting people who are the best seen so far.

Now suppose the current person *is* the best among the first k people seen so far. Let q_k be the optimal probability of eventually selecting the globally best person if we reject the k -th person and continue optimally afterwards.

If we instead accept the current person immediately, what is the probability of success?

Since the ordering of the n people is uniformly random, each of the first k positions is equally likely to contain the globally best person. Conditioned on the fact that the current person is the best among the first k , the probability that they are actually the globally best person equals:

$$\frac{k}{n}$$

Therefore:

- accepting gives success probability $\frac{k}{n}$,
- rejecting gives success probability q_k .

Hence the optimal decision is to accept iff $\frac{k}{n} \geq q_k$ ⁹.

Now observe that $\frac{k}{n}$ is increasing with k .

On the other hand, q_k is decreasing with k . To see this, note that from time k , one valid strategy is:

“Reject one more person no matter what, and then behave optimally from time $k+1$ onwards.”

This strategy achieves success probability exactly q_{k+1} . Since q_k is the *optimal* achievable success probability starting from time k , we must have:

$$q_k \geq q_{k+1}$$

Therefore there exists some threshold $0 \leq r \leq n$ such that:

- for $k \leq r$, rejecting is optimal,
- for $k > r$, accepting the first “best-so-far” person is optimal.

Hence every optimal strategy must be a threshold strategy. Note $r=0$ corresponds to accepting the first person, and $r=n$ corresponds to rejecting everyone and ending up with no partner. Both of these are valid threshold strategies, though they are not very good ones¹⁰.

⁹This is a common theme in **online algorithms** - the optimal decision at each step is determined by comparing the value of accepting now to the expected value of rejecting and continuing optimally.

¹⁰Though it may be argued that having no partner is the best choice - this is beyond the scope of this book and is left as an exercise for the reader.

3.3.2 Finding the Optimal Threshold

We now analyse the threshold strategy:

Reject the first r people, then accept the first person afterwards who is better than everyone seen so far.

Suppose the globally best person appears at position k .

For our strategy to succeed:

1. we must have $k > r$,
2. among the first $k - 1$ people, the best person must appear within the first r positions.

The first condition ensures that we do not automatically reject the best person.

The second condition ensures that no earlier candidate after position r triggers acceptance before the globally best person appears.

Now:

$$\Pr[\text{best person appears at position } k] = \frac{1}{n}$$

Further, among the first $k - 1$ people, each position is equally likely to contain their maximum. Therefore:

$$\Pr[\text{maximum among first } k - 1 \text{ lies in first } r] = \frac{r}{k - 1}$$

Hence:

$$\begin{aligned} \Pr[\text{success}] &= \sum_{k=r+1}^n \frac{1}{n} \cdot \frac{r}{k - 1} \\ &= \frac{r}{n} \sum_{k=r}^{n-1} \frac{1}{k} \end{aligned}$$

Using the approximation:

$$\sum_{k=r}^{n-1} \frac{1}{k} \approx \ln\left(\frac{n}{r}\right)$$

we obtain:

$$P(r) \approx \frac{r}{n} \ln\left(\frac{n}{r}\right)$$

Let:

$$x = \frac{r}{n}$$

Then:

$$P(x) = x \ln\left(\frac{1}{x}\right)$$

Differentiating:

$$P'(x) = \ln\left(\frac{1}{x}\right) - 1$$

Setting:

$$P'(x) = 0$$

gives:

$$\begin{aligned} \ln\left(\frac{1}{x}\right) &= 1 \\ x &= \frac{1}{e} \end{aligned}$$

Thus the optimal strategy is:

Reject approximately the first $\frac{n}{e}$ people, then accept the first person afterwards who is better than everyone seen so far.

The corresponding success probability is:

$$\boxed{\frac{1}{e}}$$

which is approximately 36.8%. Not bad at all!

Strictly speaking, we should also verify that this critical point indeed corresponds to a maximum rather than a minimum. One way is via the second derivative test:

$$P''(x) = -\frac{1}{x} < 0$$

for all $x > 0$, so $P(x)$ is concave and hence the critical point is necessarily a global maximum.

Alternatively, we can avoid calculus entirely by inspecting the behaviour near the boundaries. Observe:

$$P(x) = x \ln \left(\frac{1}{x} \right)$$

As $x \rightarrow 0$,

$$P(x) \rightarrow 0$$

since x decays faster than $\ln(1/x)$ grows.

Similarly, as $x \rightarrow 1$,

$$P(x) \rightarrow 0$$

because:

$$\ln(1) = 0$$

Since $P(x)$ is positive for $0 < x < 1$, rises away from 0, and eventually falls back to 0, the unique critical point at:

$$x = \frac{1}{e}$$

must correspond to the global maximum.

3.3.3 Behaviour for Small n

We used some limiting n approximations in finding the optimal threshold (I hope you noticed!). In case you were not planning on dating a tending to infinite number of people, you might be interested in the optimal strategy for small values of n . For finite n , the exact success probability for threshold r is:

$$P(r) = \frac{r}{n} \sum_{k=r}^{n-1} \frac{1}{k}$$

Computationally we can now easily find max for each n :

n	2	3	4	5	6	7
n/e	0.74	1.10	1.47	1.84	2.21	2.58
$\text{round}(n/e)$	1	1	1	2	2	3
Actual optimal r	1	1	1	2	2	3

The corresponding optimal success probabilities are:

n	2	3	4	5	6	7
$\text{Pr}[\text{success}]$	0.500	0.500	0.458	0.433	0.428	0.414

We observe that the optimal threshold tracks the value suggested by: $\text{round}(\frac{n}{e})$ quite closely even for relatively small values of n .

4 My Examples

4.1 Matrix Problem

Problem¹¹ Statement: Given two $n \times m$ matrices A and B , determine if they are equivalent under row and column permutations. That is, can we permute the rows and columns of A to obtain B ?

Additional Constraints: A and B both satisfy – the set of all entries is $\{1, 2, \dots, nm\}$.

Solution: First we reduce the problem to a simpler one. If we consider $R(M)$ to be the set of sets of values in the rows of a matrix M , and similarly $C(M)$ then the matrices A and B are equivalent if and only if $R(A) = R(B)$ and $C(A) = C(B)$. This reduced problem can be solved by two approaches:

4.1.1 Method 1: Set of Sets Approach (Row and Column sets)

Formally, let

$$R_i = \{a_{i1}, a_{i2}, \dots, a_{im}\}$$

be the representation of row i . We compare:

$$\{R_1, R_2, \dots, R_n\}_A = \{R_1, R_2, \dots, R_n\}_B$$

and similarly for columns.

This method is conceptually exact but computationally heavy due to the need to compare complex set-of-sets structures. Set structures can be implemented using balanced binary trees in $\Theta(n \log n)$ time - In C++ we have the `set` data structure which is implemented as a balanced binary tree. Comparing two sets of size m takes $\Theta(m \log m)$ time, and we have to do this for n rows and n columns, leading to a total time complexity of $\Theta(nm \log n \log m)$.

C++ implementation:

```
//Matrices are 0-indexed and stored in 2D vectors "first" and "second"
set<set<int>>F; //set of sets for first matrix
for(int i=0; i<n; i++)
{
    set<int>temp(first[i].begin(), first[i].end()); //set of ith row
    F.insert(temp);
}

for(int i=0; i<n; i++)
{
    set<int>temp(second[i].begin(), second[i].end()); //set of ith row
    if (!F.count(temp)) //early exit if this row doesn't exist in the first matrix
    {
        cout<<"NO\n";
        return;
    }
}
```

¹¹This question was taken from Codeforces 1980E

4.1.2 Method 2: Hashing Based Verification of Rows and Columns

To obtain a more efficient solution, each row is hashed to a pair of values (sum, squaresum), where sum and squaresum denote the sum and sum of squares of elements in the row respectively. This representation is much easier to compare across rows.

Assuming the hash is sufficiently collision-resistant, we compare the sets of row hashes for both matrices. This can be implemented efficiently using `unordered_set` in C++, which provides expected $O(1)$ insertion and lookup time.

For each row i , we compute:

$$h(R_i) = \left(\sum_{j=1}^m a_{ij}, \sum_{j=1}^m a_{ij}^2 \right)$$

We define a custom hash for a pair (x, y) as:

$$H(x, y) = x \cdot 2^{32} + y$$

where x and y are the computed row statistics. This choice of pair hash is popular for 32 bit integers, as it combines the two values into a single 64-bit integer guaranteeing unique hashes.

We insert the hashes of all rows of matrix A into an unordered set and then check if the hashes of rows of matrix B are present in this set. If any hash from B is not found in the set from A , we can conclude that the matrices are not equivalent under row permutations.

Similarly for columns, we compute the column hashes and compare them in the same manner.

This method has an expected time complexity of $\Theta(nm)$ due to the linear time required to compute the hashes and the expected constant time for hash set operations.

This method involved hashing so the correctness of the algorithm is itself a random variable – but given the constraint that the matrix entries are from $\{1, 2, \dots, nm\}$, it seems highly unlikely that this will fail. In fact, it seems that this may be a unique hash in this case, but i am yet to find a proof for general n, m C++ implementation:

```
auto pair_hash=[](int x,int y){
    return ((x)<<32)+y;
};

unordered_set<long long>s;
loop(i,n)
{
    long long v1=0,v2=0;
    loop(j,m)
    {
        int x=first[i][j];
        v1+=x;
        v2+=x*x;
    }
    s.insert(pair_hash(v1,v2));
}
loop(i,n)
{
    long long v1=0,v2=0;
    loop(j,m)
    {
```

```

        int x=second[i][j];
        v1+=x;
        v2+=x*x;
    }
    if(!s.count(pair_hash(v1,v2))) //early exit if this row doesn't exist in the first matrix
    {
        cout<<"NO\n";
        return;
    }
}

```

4.2 Rolling Window Multiset

Problem Statement: Two integer arrays x and y , each of size n , are given along with an integer k ($1 \leq k \leq n$). For every contiguous subarray (window) of size k , determine whether the corresponding windows in x and y contain the same multiset of elements at every position.

More precisely, for every index (1-based) i such that $1 \leq i \leq n - k + 1$, we compare the multisets:

$$\{x_i, x_{i+1}, \dots, x_{i+k-1}\} \quad \text{and} \quad \{y_i, y_{i+1}, \dots, y_{i+k-1}\}$$

and check whether they are identical for all i .

Rolling Hash Method

To obtain an efficient solution, we maintain two rolling hashes for each window:

$$S = \sum a_i, \quad Q = \sum a_i^2$$

These values can be updated in $\mathcal{O}(1)$ time when the window slides by removing the outgoing element and adding the incoming element.

For each position i , we compute:

$$(S_x(i), Q_x(i)) \quad \text{and} \quad (S_y(i), Q_y(i))$$

and check whether:

$$(S_x(i), Q_x(i)) = (S_y(i), Q_y(i))$$

If the condition holds for all i , we conclude that every corresponding window has identical multiset structure.

C++ Implementation

```

long long sumX=0,sumY=0;
long long sumX2=0,sumY2=0;

for(int i=0;i<k;i++)
{
    sumX+=x[i];
    sumY+=y[i];
    sumX2+=x[i]*x[i];
    sumY2+=y[i]*y[i];
}

bool ok=1;

```

```

for(int i=k;i<n;i++)
{
    if(!((sumX==sumY)&&(sumX2==sumY2)))
    {
        ok=0;
        break;
    }

    sumX-=x[i-k];
    sumX+=x[i];
    sumX2-=x[i-k]*x[i-k];
    sumX2+=x[i]*x[i];

    sumY-=y[i-k];
    sumY+=y[i];
    sumY2-=y[i-k]*y[i-k];
    sumY2+=y[i]*y[i];
}
cout<<(ok?"YES":"NO");

```

This method is very easy to implement and runs in $\mathcal{O}(n)$ time, which is efficient for this problem.

However, it relies on the assumption that the hash values are unique for different multisets.

We can improve this (if needed) by doing a second round of hashing with a different hash function like polynomial hashing.

5 Heuristic Algorithms

Many computational problems of practical interest have a statement like “Given a graph find a set of vertices of size atmost x satisfying property P .”

Now one way to guarantee a solution is to find the minimum sized set of vertices satisfying P , but this is usually not doable in polynomial time.

So we are often forced to settle for a solution that is not necessarily optimal but can be found efficiently. Such algorithms are *heuristic algorithms*: algorithms designed to quickly find solutions that are often good in practice, even if they do not always provide provable optimality guarantees or even any guarantees at all.

Inherently the analysis of heuristic algorithms involve probability and expectation, making them a natural application of the tools we have developed so far.

5.1 The Dominating Set Problem

Problem Statement: Given a graph:

$$G = (V, E)$$

find a smallest possible subset:

$$D \subseteq V$$

such that every vertex in the graph is either:

- contained in D , or
- adjacent to some vertex in D .

Such a set is called a *dominating set*.

Intuitively, we may think of the vertices in D as “control centers” or “facilities” such that every location in the graph is either itself a facility or directly connected to one.

Unfortunately, finding the minimum dominating set is NP-hard, so we want to find a polynomial time algorithm that gives us a reasonably small¹² dominating set.

5.1.1 A Randomised Solution

Suppose the graph has minimum degree d . We now consider the following extremely simple randomised strategy:

Choose k vertices randomly out of the n vertices (uniformly among the $\binom{n}{k}$ possible choices). If the chosen set forms a dominating set, output it. Otherwise, declare failure.

At first glance, this algorithm seems dumb. However, it turns out that if we choose k large enough, the probability of failure becomes small. The key question thus becomes:

What is the probability of failure as a function of k , n , and d ?

We now analyse this probability.

Probability That a Vertex Remains Undominated

Fix a vertex: $v \in V$

Since the graph has minimum degree d , $N[v]$ contains at least $d + 1$ vertices.

For v to remain undominated, none of the selected k vertices may lie inside $N[v]$

When choosing vertices uniformly at random, the probability that a single chosen vertex does *not* belong to $N[v]$ is at most:

$$1 - \frac{d+1}{n}$$

$$\implies \Pr[v \text{ remains undominated}] \leq \left(1 - \frac{d+1}{n}\right)^k$$

Using:

$$1 - x \leq e^{-x}$$

we obtain:

$$\Pr[v \text{ remains undominated}] \leq e^{-k(d+1)/n}$$

Bounding the Failure Probability

The algorithm fails if at least one vertex remains undominated.

Applying the union bound¹³

$$\Pr[\text{failure}] \leq n \cdot e^{-k(d+1)/n}$$

We would like this probability to be strictly less than 1. Therefore we require:

$$ne^{-k(d+1)/n} < 1$$

¹²We care more about the asymptotic behaviour of the size of set produced by the algorithm and less about which polynomial time complexity the algorithm has

¹³The *union bound* states that for any events: $A_1, A_2, \dots, A_n$ we always have: $\Pr[\bigcup_{i=1}^n A_i] \leq \sum_{i=1}^n \Pr[A_i]$

Taking natural logarithms:

$$\ln n - \frac{k(d+1)}{n} < 0$$

Rearranging:

$$k > \frac{n \ln n}{d+1}$$

Thus, choosing:

$$k = \frac{cn \ln n}{d+1}$$

for any constant:

$$c > 1$$

gives:

$$\Pr[\text{failure}] \leq n^{1-c}$$

which tends to 0 as $n \rightarrow \infty$.

What Does This Actually Mean?

The important point is not merely that the algorithm succeeds with high probability.

Rather, we have shown something stronger:

There exists a dominating set¹⁴ of size roughly:

$$\frac{n \log n}{d}$$

Indeed, if no such dominating set existed, our algorithm could never possibly succeed.

Thus a simple probabilistic argument simultaneously:

- proves the existence of a reasonably small dominating set,
- and gives a randomised algorithm capable of finding one.

Amplifying the Success Probability

Even if a single run fails, we may simply repeat the algorithm independently.

If the failure probability of one run is $p = \frac{1}{n^{c-1}}$, then after t independent repetitions, the probability that *every* run fails equals p^t which decreases exponentially fast.

So the tradeoff is choosing a larger c has a smaller expected number of repetitions ($\frac{n^{c-1}}{n^{c-1}-1}$)¹⁵ but a larger dominating set, while choosing a smaller c has a larger expected number of repetitions but a smaller dominating set.

¹⁴This method has a limitation - if $d < c \ln n$, then the size of the dominating set produced by the algorithm is basically the whole set of vertices, making it useless.

¹⁵Since the probability of failure is $p = (n^{c-1} - 1)/n^{c-1}$ and this is a bernoulli trial, the expected number of trials for success is $1/p$

5.1.2 A Greedy Heuristic

Another very natural approach is the following greedy algorithm:

Repeatedly select the vertex that dominates the largest number of currently undominated vertices.

More precisely:

1. Initially, all vertices are marked undominated.
2. At each step, choose a vertex whose closed neighbourhood $N[v]$ contains the largest number of undominated vertices.
3. Add this vertex to the dominating set.
4. Mark all vertices in $N[v]$ as dominated.
5. Repeat until every vertex becomes dominated.

This heuristic is extremely intuitive: at every step we greedily maximise immediate progress.

Analysing the Greedy Heuristic

Let OPT denote the size of a minimum dominating set.

Suppose that at some stage of the algorithm there remain r undominated vertices.

Since the optimal solution uses only OPT vertices to dominate the entire graph, those same OPT vertices must also collectively dominate these remaining r vertices.

Therefore, by the pigeonhole principle, at least one of those optimal vertices must dominate at least $\frac{r}{\text{OPT}}$ of the currently undominated vertices.

But the greedy algorithm always chooses a vertex dominating as many undominated vertices as possible. Hence the greedy step must dominate at least $\frac{r}{\text{OPT}}$ new vertices.

Thus after one greedy step, the number of remaining undominated vertices becomes at most

$$r - \frac{r}{\text{OPT}} = r \left(1 - \frac{1}{\text{OPT}}\right)$$

If r_t denotes the number of undominated vertices after t greedy steps, then

$$r_{t+1} \leq r_t \left(1 - \frac{1}{\text{OPT}}\right)$$

Since initially $r_0 = n$, we obtain

$$r_t \leq n \left(1 - \frac{1}{\text{OPT}}\right)^t$$

We would like all vertices to become dominated, i.e. $r_t < 1$.

Since

$$r_t \leq n \left(1 - \frac{1}{\text{OPT}}\right)^t$$

it suffices to find the smallest t such that

$$n \left(1 - \frac{1}{\text{OPT}}\right)^t < 1$$

Rearranging:

$$\left(1 - \frac{1}{\text{OPT}}\right)^t < \frac{1}{n}$$

Taking logarithms:

$$t \ln \left(1 - \frac{1}{\text{OPT}}\right) < -\ln n$$

Since $\ln(1 - \frac{1}{\text{OPT}}) < 0$, dividing reverses the inequality:

$$t > \frac{\ln n}{-\ln \left(1 - \frac{1}{\text{OPT}}\right)}$$

Let S denote the size of the dominating set produced by the greedy algorithm. Then S is the smallest integer t satisfying the above inequality. Thus:

$$S = \left\lceil \frac{\ln n}{-\ln \left(1 - \frac{1}{\text{OPT}}\right)} \right\rceil \leq \frac{\ln n}{-\ln \left(1 - \frac{1}{\text{OPT}}\right)} + 1$$

Now using the inequality

$$-\ln(1 - x) \geq x$$

with $x = \frac{1}{\text{OPT}}$, we obtain

$$-\ln \left(1 - \frac{1}{\text{OPT}}\right) \geq \frac{1}{\text{OPT}}$$

Taking reciprocals reverses the inequality:

$$\frac{1}{-\ln \left(1 - \frac{1}{\text{OPT}}\right)} \leq \text{OPT}$$

Therefore:

$$S \leq \ln n \cdot \frac{1}{-\ln \left(1 - \frac{1}{\text{OPT}}\right)} + 1$$

$$\implies S \leq \text{OPT} \ln n + 1$$

Hence the greedy algorithm produces a dominating set of size at most

$$\boxed{\text{OPT} \ln n + 1}$$

5.2 Exam Scheduling

Problem Statement: We are given a set of exams and a set of students. Each student is enrolled in a subset of the exams. We wish to schedule as many exams as possible in the same time slot, subject to the constraint that no student should have two exams scheduled at the same time¹⁶.

Equivalently, if two exams share at least one student, they cannot be scheduled together.

The goal is to find a largest possible subset of exams that can be scheduled simultaneously without any conflict.

¹⁶Or you could just give them a Time Turner and hope for the best :)

Graph Theoretic Formulation

We model the problem using a graph $G = (V, E)$ where:

- each vertex represents an exam,
- an edge exists between two vertices if there is at least one student enrolled in both corresponding exams.

In this formulation, a feasible schedule corresponds to a set of vertices with no edges between them. Such a set is called an *independent set*.

Thus, the problem reduces to finding a maximum independent set in a graph.

5.2.1 A Probabilistic Method Analysis

V is the set of all vertices. Maintain a set - the independent set we are building (S). Pick a random vertex from V - delete it from V and if none of its neighbours are in S , put it in S . Keep doing this until V is empty. This guarantees forming a valid independent set.

Note that the order in which we pick vertices is a uniformly random permutation of the vertices.

Probability a Vertex is Selected

Fix a vertex v . Consider the set consisting of v and its neighbours $N(v)$. There are $d(v) + 1$ vertices in total.

Since all relative orderings are equally likely, the probability that v is the picked earliest among these vertices is:

$$\Pr[v \in S] = \frac{1}{d(v) + 1}$$

Expected Size of the Independent Set

Let I_v be the indicator random variable for the event $v \in S$. Then:

$$|S| = \sum_{v \in V} I_v$$

Taking expectation and using linearity:

$$\mathbb{E}[|S|] = \sum_{v \in V} \mathbb{E}[I_v] = \sum_{v \in V} \frac{1}{d(v) + 1}$$

Therefore, there exists an independent set of size at least:

$$\sum_{v \in V} \frac{1}{d(v) + 1}$$

Denoting the maximum independent set size by $\alpha(G)$, we have:

$$\alpha(G) \geq \sum_{v \in V} \frac{1}{d(v) + 1}$$

Using the AM-HM inequality, we have:

$$\sum_{v \in V} \frac{1}{d(v) + 1} \geq \frac{n}{d + 1}$$

where d is the average degree of the graph. Hence:

$$\alpha(G) \geq \frac{n}{d + 1}$$

5.2.2 A Local Random Decision Method

We now describe an alternative randomized construction that produces a large independent set via local random choices.

Each vertex independently chooses a random value:

$$X_v = \begin{cases} 1 & \text{with probability } p \\ 0 & \text{with probability } 1 - p \end{cases}$$

We construct a set S by including a vertex v if:

- $X_v = 1$, and
- for every neighbour $u \in N(v)$, $X_u = 0$.

Probability of Inclusion

Fix a vertex v . The probability that v is included in S is:

$$\Pr[v \in S] = p(1 - p)^{d(v)}$$

Therefore:

$$\mathbb{E}[|S|] = \sum_{v \in V} p(1 - p)^{d(v)}$$

Guarantee on Expected Size

We analyze the randomized construction where each vertex is selected independently with probability p , and a vertex v is included in the independent set S if it is selected and none of its neighbors are selected. Then:

$$\mathbb{E}[|S|] = \sum_{v \in V} p(1 - p)^{d(v)}.$$

Using convexity of $f(x) = (1 - p)^x$ and Jensen's inequality, we obtain:

$$\sum_{v \in V} (1 - p)^{d(v)} \geq n(1 - p)^d,$$

where $d = \frac{1}{n} \sum_{v \in V} d(v)$ is the average degree. Hence:

$$\mathbb{E}[|S|] \geq np(1 - p)^d.$$

We now optimize the function:

$$f(p) = p(1 - p)^d.$$

Differentiating:

$$f'(p) = (1 - p)^d - pd(1 - p)^{d-1} = (1 - p)^{d-1}((1 - p) - pd).$$

Setting $f'(p) = 0$, we obtain:

$$1 - p - pd = 0 \quad \Rightarrow \quad p = \frac{1}{d + 1}.$$

Substituting this optimal value gives:

$$\mathbb{E}[|S|] \geq n \cdot \frac{1}{d + 1} \left(1 - \frac{1}{d + 1}\right)^d = \frac{n}{d + 1} \left(\frac{d}{d + 1}\right)^d.$$

Using the standard bound $\left(1 - \frac{1}{d+1}\right)^d \geq e^{-d/(d+1)}$, we obtain:

$$\mathbb{E}[|S|] \geq \frac{n}{d + 1} \cdot e^{-d/(d+1)} \geq \frac{n}{e(d + 1)}.$$

So we would expect to need to repeat this process about $e(d + 1)$ times to schedule all exams.